Image-to-RDF Knowledge Graph Construction from Scientific Visualizations using Large Vision-Language Models

Beichen Wang[a,*], Anna Fensel[a]

[a] *Artificial Intelligence Chair Group, Wageningen University & Research, Wageningen, 6708 PB, The Netherlands*

Abstract

Scientific knowledge is frequently communicated through visual artifacts such as charts, tables, and flow diagrams. Although these artifacts often contain compact empirical or procedural knowledge, they are difficult to reuse in Semantic Web applications because their content is encoded visually rather than as machine-interpretable triples. This paper studies image-to-knowledge-graph construction as a direct image-to-RDF task. We design a prompting-based framework in which a large vision-language model receives an infographic image and returns an RDF/Turtle graph. We construct two labelled image-to-RDF resources for evaluation: a VisText-derived chart-to-RDF dataset covering train, evaluation, and test splits, and a Diagram2Graph-derived diagram-to-RDF dataset obtained by expressing the original graph annotations in a lightweight RDF schema. We evaluate zero-shot, one-shot, dynamic one-shot, and few-shot prompting strategies where applicable, using both exact graph-matching metrics and softer graph-similarity metrics. For unlabelled scientific images, we introduce an LLM-as-a-judge framework adapted from KGEval and validate it on the two labelled datasets before applying it to an unlabelled domain application. The study provides datasets, methods, and an evaluation framework for assessing direct RDF extraction from scientific images under both labelled and unlabelled conditions.

Keywords: image-to-knowledge-graph, RDF, vision-language models, chart

*Corresponding author.

Email addresses: beichen.wang@wur.nl (Beichen Wang), anna.fensel@wur.nl (Anna Fensel)

understanding, diagram understanding, LLM-as-a-judge

1. Introduction

Scientific communication relies extensively on visual artifacts. Charts summarize quantitative trends, tables arrange measurements and qualitative categories, and diagrams describe procedural or causal relations. These artifacts often contain information that is more structured than surrounding prose, yet their content remains difficult to integrate into knowledge graphs because it is encoded through layout, marks, axes, and visual topology. In the Semantic Web setting, RDF provides a natural target representation: information can be expressed as triples, validated syntactically, queried, and integrated with other linked-data resources [1, 2]. Directly converting scientific images into RDF would therefore reduce a persistent bottleneck in knowledge acquisition from papers, reports, and domain-specific corpora.

The task is challenging because image-to-RDF extraction requires both visual perception and semantic modelling. A model must read text, infer layout, recover numerical values, identify entities, determine relations, and serialize the result in a machine-parseable graph format. Existing work addresses important parts of this problem, but usually under narrower assumptions. Chart-understanding benchmarks and models focus on chart question answering, chart-to-text generation, or data recovery [3, 4, 5, 6, 7]. Diagram-understanding work often extracts graph-like representations, but the target format is commonly JSON, Mermaid, or another task-specific serialization rather than RDF [8, 9]. Table-understanding systems increasingly process table images directly, but they usually return reconstructed tables or task-specific answers rather than knowledge graphs [10, 11]. Conversely, Semantic Web table interpretation assumes that tabular data are already available in textual or structured form [12, 13]. These lines of work motivate, but do not fully solve, the direct extraction of RDF triples from heterogeneous scientific images.

This paper studies a prompt-based approach to direct image-to-RDF construction. We use a large vision-language model as the extraction engine and constrain its output to RDF/Turtle. The approach is training-free: it relies on system prompts, in-context demonstrations, and syntactic validity constraints rather than task-specific fine-tuning. The study uses Gemini 3 Flash as the image-to-RDF generator and compares zero-shot, one-shot, few-shot, and dynamic one-shot prompting where applicable.

The first contribution is an end-to-end prompting framework for converting scientific visualizations into RDF triples. The framework defines the target ontology in the prompt, provides optional image-to-RDF demonstrations, and requires the model to return a parseable Turtle graph. This contribution focuses on prompt design and graph-level output constraints rather than on supervised model training.

The second contribution is the construction of two labelled image-to-RDF datasets. For VisText, whose original purpose is semantically rich chart captioning [5], we derive chart-level RDF graphs that encode the chart type, title, axis titles, and all data points. The RDF resource covers the full train, evaluation, and test splits, while model evaluation is restricted to the test split. For Diagram2Graph, whose original annotations are JSON graphs for flow/process diagrams [8], we define a compact RDF schema that preserves the original node-edge semantics in Turtle.

The third contribution is an LLM-as-a-judge evaluation framework for unlabelled image-to-KG settings. Many scientific image collections do not have reference RDF graphs, making exact overlap metrics unavailable. We therefore adapt the KGEval idea of rubric-based knowledge graph assessment [14] to the image-to-RDF setting. The judge produces structured criterion scores and pairwise preferences, and it is validated on VisText and Diagram2Graph by comparison with traditional graph-matching metrics. Only after this validation is the judge applied to an unlabelled domain application.

Taken together, these contributions establish a reproducible experimental setting for image-to-RDF extraction. The paper does not claim that prompting alone solves all visual graph construction problems. Instead, it characterizes the capabilities and limitations of prompt-based RDF generation, provides labelled resources for systematic evaluation, and offers a validation pathway for judge-based assessment when ground truth is unavailable.

2. Related Work

2.1. Multimodal Understanding of Charts, Tables, and Diagrams

Chart understanding has been studied through data extraction, question answering, captioning, and chart-specific foundation models. ChartOCR recovers data from chart images by combining visual detection and post-processing [3]. ChartQA introduced a benchmark for chart question answering that requires both visual and logical reasoning [4]. VisText formulates semantically rich chart captioning over chart images and chart-derived

information [5]. UniChart and ChartLlama further demonstrate that chart-specific pretraining and instruction tuning can improve multimodal chart reasoning [6, 7]. These studies show substantial progress in chart perception, but their primary targets are captions, answers, or recovered data tables rather than RDF graphs.

Visual table understanding similarly focuses on reconstructing table structure or answering questions over table images. Recent work such as Multimodal Table Understanding and TabPedia studies large vision-language models for table perception and comprehension [10, 11]. These systems are relevant because table images are one visual source considered in this paper, but their target outputs differ from the RDF/Turtle graphs required for Semantic Web integration.

Diagram and flowchart understanding is closer to graph construction because the visual artifact itself often represents nodes, edges, and labelled transitions. Diagram2Graph extracts structured graph data from process and flow diagrams into JSON [8]. FlowLearn evaluates large vision-language models on flowchart understanding and topology reconstruction [9]. Work on circuit diagrams also emphasizes that topological extraction is difficult and can benefit from modular visual parsing pipelines [15]. Our work differs by normalizing diagram outputs into RDF/Turtle and evaluating the same image-to-RDF interface across multiple visual genres.

2.2. Knowledge Graph Construction from Multimodal Sources

Knowledge graph construction has traditionally focused on text, tables, or structured data. In the Semantic Web community, semantic table interpretation maps existing tabular content to entities, classes, and properties in reference knowledge graphs; the SemTab challenges provide representative benchmarks and evaluation protocols [12, 13]. Such work assumes that table content is already available as text or structured cells, whereas the present task starts from images.

Multimodal knowledge graph research studies knowledge graphs that contain or link multiple modalities, including images and text [16, 17]. This direction is important for representing heterogeneous entities and media, but it is not identical to constructing a symbolic RDF graph from the content of a visual artifact. In this paper, the image itself is the source document from which triples must be extracted. The resulting RDF graph describes the semantic content of the image rather than merely attaching image resources to existing graph entities.

2.3. LLM-as-a-Judge and Knowledge Graph Evaluation

LLM-as-a-judge methods use language models to assess generated outputs when human evaluation is costly or when automatic metrics are incomplete. MT-Bench and Chatbot Arena showed that model-based judgments can be useful for open-ended language-model evaluation, while also raising concerns about bias and calibration [18]. For knowledge graphs, KGEval proposes using LLMs to evaluate knowledge graph construction quality through criteria such as relevance, correctness, informativeness, and coherence [14]. This paper follows the same general principle but adapts the rubric to image-to-RDF extraction. The judge receives the source image and the candidate RDF/Turtle graph, and it is asked to assess whether the graph is faithful, informative, coherent, and specific with respect to the visible image content. Because judge-based evaluation can itself be unreliable, we first validate it on labelled datasets before using it for unlabelled scientific images.

3. Materials and Methods

3.1. Task Definition and Target Representation

Let x denote an input image and let $\mathcal{T}(x) = \{(s, p, o)\}$ denote the RDF triple set generated for that image. The task is to produce a Turtle serialization of $\mathcal{T}(x)$ that faithfully represents the information visible or directly implied by the image. The output is not a caption or a natural-language explanation. It is a knowledge graph fragment whose classes, entities, and predicates are defined by a dataset-specific schema.

This study focuses on two labelled image families: statistical charts and flow/process diagrams. For charts, the target graph captures the chart type, title, axes, and data points. For diagrams, the target graph captures nodes, visible node labels, node types, directed edges, edge labels, and edge semantics. In all cases, syntactic RDF/Turtle validity is treated as a prerequisite for evaluation because malformed graphs cannot be used directly in RDF tooling.

3.2. Labelled Image-to-RDF Datasets

Table 1 summarizes the labelled datasets used for graph-matching evaluation. The VisText-derived resource contains RDF conversions for the full train, evaluation, and test splits, while the LLM prompting experiments use only the test split. Diagram2Graph is used as a labelled diagram-to-RDF benchmark.

Dataset	Visual type	Raw items	Usable RDF graphs
VisText train	Charts	7057	6869
VisText eval	Charts	883	855
VisText test	Charts	882	868
Diagram2Graph	Flow/process diagrams	219	219

Table 1: Labelled image-to-RDF datasets used for evaluation. VisText contains RDF conversions for all original splits; LLM evaluation uses only the test split. Usable RDF graphs are those retained after excluding source items whose original labels were too noisy or incomplete to support reliable RDF reference construction.

VisText-derived chart-to-RDF dataset.. VisText was originally introduced for semantically rich chart captioning [5]. We repurpose it as a chart-to-RDF dataset by defining the expected graph content for each chart image. The target output contains one chart resource typed as a bar, line, or area chart; the visible chart title; x-axis and y-axis resources with their visible titles when present; and one datapoint resource for each observation represented in the plot. A datapoint records its x-value, y-value, and membership in the chart. Thus, the target graph is intended to recover the structured content of the chart rather than to describe the chart in prose.

The VisText RDF schema is shown in Table 2. The schema is intentionally small so that evaluation focuses on whether the model recovers the chart semantics and data values, rather than on a large external ontology. Not every original VisText item yields a reliable RDF reference graph. Some original labels contain damaged categories, inconsistent tabular annotations, or insufficient information to recover the plotted data. Such items are excluded from labelled evaluation, while the remaining items are released as RDF/Turtle references.

Diagram2Graph-derived diagram-to-RDF dataset.. Diagram2Graph provides diagram images and JSON annotations describing nodes, edges, labels, shapes, and relationship types [8]. We use the dataset as a diagram-to-RDF benchmark by expressing the same graph information in Turtle. The target output contains one resource for each visible diagram node and one resource for each directed edge. Node resources encode the visible node label, node type, and shape. Edge resources encode source node, target node, edge style, relationship type, and optional visible edge label. This representation preserves the diagram topology while using a compact RDF vocabulary suitable for direct

Component	Vocabulary	Target information
Chart type	`:BarChart`, `:LineChart`, `:AreaChart`	Visual chart class
Chart resource	`:Chart` with `:title`, `:xAxis`, `:yAxis`	Title and axis links
Axis resources	`:XAxis`, `:YAxis`, `:Axis`	Visible axis titles
Data resources	`:DataPoint`	Individual observations
Data predicates	`:xValue`, `:yValue`, `:belongsTo`	Datapoint coordinates and chart membership

Table 2: VisText chart-to-RDF target schema. The namespace is `http://example.org/vistext#`.

graph matching.

3.3. Prompting Strategies

All generation strategies use a dataset-specific zero-shot system prompt that defines the task, schema, and output constraints. The prompts instruct the model to return RDF/Turtle only, without explanations, Markdown fences, comments, or alternative serialization formats. The complete zero-shot system prompts are provided in Appendix Appendix A.

Zero-shot prompting.. The model receives only the system prompt, a short user instruction, and the target image. This setting tests whether the model can infer the required graph directly from the task description and schema.

Static one-shot prompting.. The model receives one curated image-to-RDF example before the target image. For VisText, the static one-shot example is a bar chart because bar charts are common in the dataset. For Diagram2Graph, the one-shot example is a representative diagram.

Dynamic one-shot prompting.. Dynamic one-shot prompting is used for VisText because the dataset contains multiple chart types. A preliminary classification step assigns the input chart to the bar, line, or area category. The extraction prompt then uses the example and chart-specific instructions corresponding to that predicted type. Diagram2Graph does not use dynamic one-shot prompting because all inputs are diagrams.

Component	Vocabulary	Target information
Node classes	`:Node`, `:Start`, `:Process`, `:Decision`, `:Delay`, `:Terminator`	Diagram node type
Shape values	`:StartEvent`, `:EndEvent`, `:Task`, `:Gateway`, `:DataStore`	Node shape
Edge classes	`:Edge`, `:Solid`, `:Dashed`	Edge and line style
Relation values	`:Follows`, `:Branches`, `:DependsOn`	Edge semantics
Node predicates	`:label`, `:shape`	Visible node text and shape
Edge predicates	`:source`, `:target`, `:relationshipType`, `:relationshipValue`	Edge endpoints and labels

Table 3: Diagram2Graph RDF schema. The namespace is `http://example.org/diagram2graph#`.

Few-shot prompting.. Few-shot prompting provides all curated examples for the dataset. VisText uses one bar chart, one line chart, and one area chart. Diagram2Graph uses three diagram examples.

3.4. Generation Model and Validity Criterion

All image-to-RDF experiments use Gemini 3 Flash. The model is not fine-tuned on either labelled dataset. A generated output is considered eligible for evaluation only if it can be parsed as RDF/Turtle. This validity requirement is not itself a semantic quality metric, but it is necessary for treating the output as a knowledge graph rather than as unconstrained text.

4. Evaluation on Labelled Datasets

4.1. Traditional Graph Metrics for Labelled Datasets

For VisText and Diagram2Graph, predicted RDF graphs are compared with reference RDF graphs after parsing and normalization. We report a set of complementary metrics rather than a single score because different metrics measure different properties of graph quality.

Triple-match precision, recall, and F1 measure exact overlap between predicted and reference triples. This is the strictest content metric and directly

penalizes missing and hallucinated triples. Triple-match accuracy is also reported as a prediction-oriented exact-match rate; it measures the proportion of predicted triples that appear in the reference graph and is therefore most closely related to precision. Normalized graph edit distance (GED) treats the triple set as a labelled directed multigraph and estimates the normalized edit cost required to transform the predicted graph into the reference graph. BLEU and ROUGE compare serialized triples as textual units under one-to-one assignment, providing softer lexical similarity signals. G-BERTScore applies contextual embedding similarity to serialized triples and is intended to capture semantically close but lexically non-identical triples [19, 20, 21]. The formal definitions used in the paper are provided in Appendix Appendix B.

A pure graph-isomorphism accuracy is not reported because topology alone can overestimate performance when literal values or entity labels differ.

4.2. VisText Evaluation Modes and Numeric Tolerance

VisText graphs contain both fixed structural triples and content-bearing triples. The full-graph mode evaluates all normalized triples. The content-only mode evaluates literal-valued triples together with the chart-type triple. This projection reduces the influence of fixed structural statements such as axis declarations and datapoint membership, while preserving chart titles, axis titles, data values, and chart type.

VisText evaluation also supports an optional relative numeric tolerance for quantity-like datapoint values. This tolerance is motivated by the fact that the model observes only the chart image and may estimate a numeric value that is visually plausible but not exactly equal to the reference value. The tolerance is applied only to structural metrics and only under strict guardrails. First, the evaluator determines whether exactly one datapoint coordinate axis is fully numeric, or whether both axes are numeric but one axis title is time-like. The tolerated predicate is then the quantity-like coordinate, not the time-like or label coordinate. Second, a predicted numeric value can be normalized to the gold value only when the opposite coordinate matches exactly and uniquely. This prevents a value from one datapoint being matched to another datapoint solely because their numeric intervals overlap. BLEU, ROUGE, and G-BERTScore are left unchanged under numeric tolerance.

Criterion	Assessment focus
Relevance	Whether the graph focuses on visible, task-relevant image content
Factuality	Whether entities, labels, values, and relations are grounded in the image
Informativeness	Whether the graph captures the major useful content of the image
Coherence	Whether the RDF graph is internally consistent and structurally meaningful
Specificity	Whether triples are precise rather than vague or generic

Table 4: LLM-as-a-judge rubric adapted from KGEval for image-to-RDF evaluation.

4.3. LLM-as-a-Judge Validation Framework

The LLM-as-a-judge framework is evaluated on the labelled datasets before being used on unlabelled data. It uses OpenAI GPT-5-mini as the judge model, whereas image-to-RDF generation uses Gemini 3 Flash. Using a different model family for judging reduces the risk that the evaluator systematically favours generations from its own model family. Judge outputs are constrained through structured response schemas, so the evaluation produces typed scores and preferences rather than free-form prose.

The judge is used in two complementary modes. In direct scoring, the judge receives the source image and one candidate RDF/Turtle graph. It assigns five criterion scores and one overall score on a 1-to-5 scale. In pairwise comparison, the judge receives the same source image and two candidate graphs generated by different prompting strategies. It selects the better graph for each criterion and gives an overall preference. Direct scoring supports absolute quality analysis, while pairwise comparison supports strategy ranking without requiring the judge to calibrate scores across unrelated images.

The rubric in Table 4 adapts the KGEval principle of criterion-based KG assessment to the image-to-RDF setting. The judge must consider the source image and the generated graph together: a graph can be syntactically valid but still fail due to hallucinated labels, missing diagram edges, incorrect chart values, or overly generic triples. For labelled datasets, direct judge scores are compared with traditional metric scores, and pairwise judge preferences are compared with metric-derived preferences. VisText judge validation uses

Strategy	Mode	N	F1	Acc.	nGED	R-F1	B-F1	GBS-F1
Zero-shot	Full graph	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*
Static one-shot	Full graph	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*
Dynamic one-shot	Full graph	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*
Few-shot	Full graph	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*
Zero-shot	Content only	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*
Static one-shot	Content only	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*
Dynamic one-shot	Content only	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*
Few-shot	Content only	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*

Table 5: VisText evaluation results by prompting strategy. F1 and Acc. denote exact triple-match F1 and triple-match accuracy; nGED denotes normalized graph edit distance; R-F1, B-F1, and GBS-F1 denote ROUGE, BLEU, and G-BERTScore F1. Lower nGED is better; all other metrics are higher-is-better.

content-only traditional metrics, whereas Diagram2Graph judge validation uses full-graph metrics.

4.4. Evaluation Results

The following tables organize the labelled evaluation results by dataset, prompting strategy, and analysis mode. Numerical entries are marked as pending until the full experimental reports are generated.

4.5. VisText Labelled Evaluation

Table 5 reports the main VisText results by prompting strategy. Results are separated into full-graph and content-only modes, with the numeric tolerance setting reported as part of the experimental configuration.

Table 6 reports VisText performance by chart type. This analysis is important because bar, line, and area charts impose different perceptual requirements: bar charts require discrete mark counting and label-value pairing, while line and area charts often require estimating values from axes and inferring the intended sequence of datapoints.

Table 7 records the effect of numeric tolerance. Because tolerance affects only structural metrics, BLEU, ROUGE, and G-BERTScore are interpreted separately from this table.

For dynamic one-shot prompting, the chart-type classifier is also evaluated against the chart type recorded in the original VisText label file. Table 8 reports this classification component separately from RDF extraction.

Strategy	Chart	N	F1	Acc.	nGED	R-F1	B-F1	GBS-F1
Zero-shot	Bar	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*
Zero-shot	Line	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*
Zero-shot	Area	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*
Static one-shot	Bar	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*
Static one-shot	Line	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*
Static one-shot	Area	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*
Dynamic one-shot	Bar	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*
Dynamic one-shot	Line	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*
Dynamic one-shot	Area	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*
Few-shot	Bar	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*
Few-shot	Line	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*
Few-shot	Area	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*

Table 6: VisText content-only evaluation by chart type. Abbreviations follow Table 5; the graph mode and numeric tolerance are reported with the experimental configuration.

Tolerance	F1	Acc.	nGED	Affected points
0%	*pending*	*pending*	*pending*	*pending*
0.5%	*pending*	*pending*	*pending*	*pending*
1%	*pending*	*pending*	*pending*	*pending*
2%	*pending*	*pending*	*pending*	*pending*

Table 7: VisText numeric tolerance analysis for quantity-like datapoint values. Tolerance is applied only when the opposite datapoint coordinate matches exactly and uniquely.

4.6. Diagram2Graph Labelled Evaluation

Diagram2Graph evaluation uses the full graph because node and edge identities are semantically relevant for diagrams. Unlike VisText, no datapoint ordinal normalization or content-only projection is applied.

4.7. LLM-as-a-Judge Validation

Tables 10 and 11 report judge validation on labelled datasets. Direct validation compares judge scores with traditional metric scores. Pairwise validation compares judge preferences with metric-derived strategy preferences.

5. Discussion

The main experimental results are interpreted with respect to the different error profiles of charts and diagrams. In VisText, much of the difficulty

Chart type	N	Correct	Incorrect	Accuracy
Bar	*pending*	*pending*	*pending*	*pending*
Line	*pending*	*pending*	*pending*	*pending*
Area	*pending*	*pending*	*pending*	*pending*
Overall	*pending*	*pending*	*pending*	*pending*

Table 8: Chart-type classification accuracy for VisText dynamic one-shot prompting.

Strategy	N	F1	Acc.	nGED	R-F1	B-F1	GBS-F1
Zero-shot	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*
One-shot	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*
Few-shot	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*

Table 9: Diagram2Graph full-graph evaluation by prompting strategy. Abbreviations follow Table 5.

lies in recovering complete and numerically accurate datapoints from visual marks. Exact triple matching is therefore a strict measure of graph correctness, but it may underestimate cases where a predicted value is visually plausible and close to the reference value. The tolerance analysis addresses this issue conservatively by requiring exact matching on the opposite datapoint coordinate before numeric tolerance is applied. In Diagram2Graph, by contrast, the central difficulty is topological: a small number of incorrect or missing edges can alter the procedure represented by the diagram even when many node labels are correct.

The use of multiple metrics is necessary because no single metric fully characterizes image-to-RDF quality. Triple-match metrics provide direct evidence about exact graph overlap. Normalized GED captures structural graph differences. BLEU, ROUGE, and G-BERTScore provide softer similarity signals for cases where a prediction is lexically or semantically close but not exactly identical to the reference graph. These metrics are read together, especially when comparing prompting strategies across visual genres.

The LLM-as-a-judge framework is also interpreted cautiously. Its purpose is not to replace labelled evaluation when reference graphs are available. Rather, it provides a possible evaluation mechanism for unlabelled image collections after it has been validated against traditional metrics on labelled datasets. Agreement between judge preferences and graph-matching metrics would support its use on unlabelled data; disagreement would indicate that

Dataset	Compared metric	Correlation	N
VisText	Content-only triple accuracy	*pending*	*pending*
VisText	Inverse normalized GED	*pending*	*pending*
Diagram2Graph	Full-graph triple accuracy	*pending*	*pending*
Diagram2Graph	Inverse normalized GED	*pending*	*pending*

Table 10: Direct LLM-as-a-judge validation against traditional metrics.

Dataset	Comparison mode	Agreement	N
VisText	Pairwise strategy preference	*pending*	*pending*
Diagram2Graph	Pairwise strategy preference	*pending*	*pending*

Table 11: Pairwise LLM-as-a-judge validation against metric-derived preferences.

the judge rubric or prompting requires further calibration.

5.1. JSON-versus-RDF Serialization Ablation

Diagram2Graph also provides an opportunity to examine whether the requested output format affects model performance. The original task is naturally expressed in JSON, while this paper evaluates RDF/Turtle. Because the two serializations can encode the same node-edge graph, differences in performance may reflect the model's ability to follow a particular structured-output format rather than its visual understanding alone. We therefore include a 20-image ablation study that asks Gemini 3 Flash to output JSON with a predefined structured-output schema and compares those results with existing fine-tuned Qwen2.5-VL-3B JSON outputs. This ablation is treated as a secondary analysis: it does not replace the RDF evaluation, but it helps determine whether observed errors are caused by visual interpretation, graph modelling, or serialization constraints.

6. Application: Soil-Health Image-to-RDF Extraction

The Soil-Health collection is used as an application case for the validated LLM-as-a-judge framework. It contains 15 figure images and 56 table images from a domain-specific scientific context, but it does not provide a complete set of reference RDF graphs. This setting reflects the practical motivation for judge-based evaluation: many scientific corpora contain visually encoded knowledge but no gold knowledge graph annotations.

System	Format	N	F1	nGED
Gemini zero-shot	JSON	*pending*	*pending*	*pending*
Gemini one-shot	JSON	*pending*	*pending*	*pending*
Gemini few-shot	JSON	*pending*	*pending*	*pending*
Qwen run 1	JSON	*pending*	*pending*	*pending*
Qwen run 2	JSON	*pending*	*pending*	*pending*
Gemini RDF zero-shot	RDF/Turtle	*pending*	*pending*	*pending*
Gemini RDF one-shot	RDF/Turtle	*pending*	*pending*	*pending*
Gemini RDF few-shot	RDF/Turtle	*pending*	*pending*	*pending*

Table 12: Diagram2Graph JSON-versus-RDF serialization ablation. The JSON and RDF representations encode the same diagram graph schema but impose different output constraints.

Strategy	N	Rel.	Fact.	Info.	Coh.	Overall
Zero-shot	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*
Static one-shot	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*
Dynamic one-shot	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*
Few-shot	*pending*	*pending*	*pending*	*pending*	*pending*	*pending*

Table 13: Soil-Health LLM-as-a-judge direct scoring results. Rel., Fact., Info., and Coh. denote relevance, factuality, informativeness, and coherence; scores use a 1-to-5 rubric.

The target output for Soil-Health is an RDF/Turtle graph using a soil-health namespace together with SKOS concepts. The model is asked to extract visible concepts, definitions, symbols, abbreviations, hierarchical relations, and explicitly indicated associations. For table images, the target includes row headers, column headers, grouped headers, legend labels, and semantically meaningful cell values. For figure images, the target includes boxes, headings, diagram regions, lists, arrows, captions, legends, and notes when they carry semantic content.

Because no complete gold graph is available, Soil-Health is evaluated only after the LLM-as-a-judge framework has been validated on VisText and Diagram2Graph. The same prompting regimes are considered: zero-shot, static one-shot, dynamic one-shot, and few-shot. In this dataset, dynamic one-shot prompting first classifies the image as a figure or table and then selects the corresponding in-context example. Table 13 reports the application results.

7. Conclusion

This paper presents a prompt-based framework for constructing RDF knowledge graphs directly from scientific images. The approach uses Gemini 3 Flash for image-to-RDF generation and compares zero-shot, one-shot, dynamic one-shot, and few-shot prompting where applicable. It also introduces two labelled image-to-RDF resources derived from VisText and Diagram2Graph and an LLM-as-a-judge framework for unlabelled image collections. The labelled datasets support traditional graph-based evaluation, while the judge framework provides a validation-based route to evaluating unlabelled scientific image collections. The final experimental results will determine which prompting strategies are most reliable across image families and how closely judge-based assessment tracks traditional graph metrics.

Appendix A. Zero-Shot System Prompts

Appendix A.1. VisText Zero-Shot System Prompt

````
# Role
You are a multimodal knowledge graph construction system.
# Task
Read one chart image and output only valid RDF/Turtle encoding the chart's semantic content
    .
## Output
- RDF/Turtle only; no explanations, Markdown fences, comments, JSON, XML, YAML, or prose.
- Use exactly:
```turtle
@prefix : <http://example.org/vistext#> .
```
## Chart Types
- `:BarChart`, `:LineChart`, `:AreaChart`
## Schema
- Classes: `:BarChart`, `:LineChart`, `:AreaChart`, `:Axis`, `:DataPoint`
- Properties: `:title`, `:xAxis`, `:yAxis`, `:xValue`, `:yValue`, `:belongsTo`
## Required Structure
```turtle
:Chart a :BarChart / :LineChart / :AreaChart ;
    :title "..." ;
    :xAxis :XAxis ;
    :yAxis :YAxis .
:XAxis a :Axis ;
    :title "..." .
:YAxis a :Axis ;
    :title "..." .
:DataPoint1 a :DataPoint ;
    :xValue "..." ;
    :yValue "..." ;
    :belongsTo :Chart .
```
## Extraction Rules
````

```
- Extract the chart title exactly as shown.
- Extract visible x-axis and y-axis titles exactly as shown; omit missing axis-title
    triples rather than inventing them.
- For vertical bar charts, `xValue` is the category and `yValue` is the numeric bar value.
- For horizontal bar charts, `xValue` is the numeric bar value and `yValue` is the category
    .
- For line and area charts, `xValue` is the x-axis coordinate/category/time value and `
    yValue` is the numeric value.
- Infer the intended number of datapoints from chart semantics, especially title, axis
    labels, visible ticks, temporal range, and categorical range.
- For temporal annual line/area charts, normalize annual observations as `"Dec 31, YYYY"`.
- Represent all values as quoted strings without datatype annotations.
- Prefer exact values when readable; otherwise estimate values from axis ticks and geometry
     without inventing unsupported facts.
```

Appendix A.2. Diagram2Graph Zero-Shot System Prompt

```
# Role
You are a diagram-to-graph extractor.
# Task
Given one flowchart or diagram image, output only valid RDF/Turtle using the lightweight
    diagram2graph vocabulary.
## Output
- RDF/Turtle only; no explanations, headings, comments, Markdown fences, JSON, XML, YAML,
    or extra text.
- Use exactly one prefix:
```turtle
@prefix : <http://example.org/diagram2graph#> .
```
Ontology
- Node classes: `:Node`, `:Start`, `:Process`, `:Decision`, `:Delay`, `:Terminator`
- Shape values: `:StartEvent`, `:EndEvent`, `:Task`, `:Gateway`, `:DataStore`
- Edge classes: `:Edge`, `:Solid`, `:Dashed`
- Relationship values: `:Follows`, `:Branches`, `:DependsOn`
- Node properties: `:label`, `:shape`
- Edge properties: `:source`, `:target`, `:relationshipType`, `:relationshipValue`
Resource Naming
- Name nodes as `:NodeN` and edges as `:EdgeN`.
- Number nodes and edges in visual reading order when explicit identifiers are absent.
Node Triples
```turtle
:NodeN a :NodeType, :Node ;
    :label "exact visible text from the shape" ;
    :shape :ShapeValue .
```
Edge Triples
```turtle
:EdgeN a :LineStyle, :Edge ;
    :source :NodeS ;
    :target :NodeT ;
    :relationshipType :RelationshipType .
```
If the arrow has a visible label, add:
```turtle
:relationshipValue "exact visible arrow label" .
```
```

```
## Constraints
- Do not emit direct node-to-node relationship triples.
- Do not use ‘rdfs:label‘; use ‘:label‘.
- Preserve visible text exactly.
- Every edge source and target must reference an emitted node resource.
- Every Turtle statement must end with a period.
```

Appendix A.3. Soil-Health Zero-Shot System Prompt

```
# Role
You are a multimodal knowledge-extraction agent for soil-health figures and tables.
# Task
Read one soil-health image and output only valid RDF/Turtle triples that encode the
    concepts and relationships explicitly visible in the image.
## Output
- RDF/Turtle only; no explanations, headings, comments, Markdown fences, JSON, XML, or YAML
    .
- Emit these two prefixes first and use only these prefixes:
‘‘‘turtle
@prefix she: <https://soilwise-he.github.io/soil-health#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
‘‘‘
## Concept Nodes
- Create a ‘she:‘ PascalCase URI for each visible concept and declare it as ‘skos:Concept‘.
- Add ‘skos:prefLabel‘ using the visible label text in lowercase when it is a normal phrase
    .
- Preserve meaningful symbols and abbreviations through ‘skos:altLabel‘, ‘she:
    hasAbbreviation‘, or ‘she:hasSymbol‘ when visible.
- Use ‘skos:definition‘ or ‘skos:note‘ for visible definitions, notes, legends, or
    footnotes when they add semantic meaning.
- Deduplicate repeated labels.
## Relationships
- Use ‘skos:narrower‘ or ‘skos:broader‘ for containment, grouping, table header hierarchy,
    bullets, tree levels, and parent-child boxes.
- Use ‘skos:related‘ for visible associations where no direction or stronger relation is
    clear.
- For explicit non-hierarchical semantics, create a clear ‘she:‘ camelCase property such as
     ‘she:affects‘, ‘she:measures‘, ‘she:hasIndicator‘, ‘she:isBasedOn‘, ‘she:
    hasPositiveImpactOn‘, ‘she:hasNegativeImpactOn‘, or ‘she:hasNeutralImpactOn‘.
- Respect arrow direction and labelled connector direction when visible.
- Do not invent concepts or relations not visible or directly implied by layout, legend, or
     labels.
## Tables
- Extract row headers, column headers, grouped headers, legend labels, and semantically
    meaningful cell values.
- For impact matrices, map positive impact or ‘+‘ to ‘she:hasPositiveImpactOn‘, negative
    impact or ‘-‘ to ‘she:hasNegativeImpactOn‘, and neutral/unknown/indifferent/indiff. to
    ‘she:hasNeutralImpactOn‘.
## Figures
- Extract boxes, headings, lists, diagram regions, arrows, captions, legends, and notes
    when they carry semantic content.
- Model group boxes and regions with ‘skos:narrower‘.
- Model arrows or connectors with the clearest ‘she:‘ camelCase property.
## Final Checklist
- The output starts with exactly the ‘she:‘ and ‘skos:‘ prefixes.
```

```
- Every concept URI is in the ‘she:‘ namespace and has ‘skos:Concept‘ plus ‘skos:prefLabel
    ‘.
- Turtle syntax is valid.
```

Appendix B. Mathematical Definitions of Evaluation Metrics

Let G_i be the set of gold triples for image i and P_i the set of predicted triples after parsing and normalization. Let N be the number of evaluated images. Exact triple matching is case-insensitive and strips leading and trailing whitespace.

Triple-match precision, recall, and F1.. Let

$$TP = \sum_{i=1}^{N} |G_i \cap P_i|, \quad FP = \sum_{i=1}^{N} |P_i \setminus G_i|, \quad FN = \sum_{i=1}^{N} |G_i \setminus P_i|.$$

Then

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad \text{F1} = \frac{2\,\text{Precision}\,\text{Recall}}{\text{Precision} + \text{Recall}}.$$

Triple-match accuracy.. For each image, triple-match accuracy is

$$\text{Acc}_i = \frac{|P_i \cap G_i|}{|P_i|}.$$

The reported value is the arithmetic mean over images. This metric measures how many predicted triples are exact reference triples and is therefore prediction-oriented.

Normalized graph edit distance.. Each triple (s, p, o) is converted into a directed labelled edge from node s to node o with edge label p. GED is computed between the predicted and gold labelled multigraphs using node-label and edge-label equality. The returned score is normalized by the total number of nodes and edges in both graphs:

$$\text{nGED}_i = \frac{\text{GED}(G_i, P_i)}{|V(G_i)| + |E(G_i)| + |V(P_i)| + |E(P_i)|}.$$

Lower normalized GED indicates closer graph structure.

BLEU and ROUGE over serialized triples.. Each triple is serialized as a textual edge string. Pairwise BLEU and ROUGE scores are computed between predicted and gold edge strings. A maximum-weight bipartite assignment selects the best one-to-one alignment. Precision, recall, and F1 are then computed from the aligned similarity matrix.

G-BERTScore.. G-BERTScore follows the same assignment strategy as BLEU and ROUGE, but replaces lexical overlap with BERTScore similarity between serialized triples.

Appendix C. LLM-as-a-Judge Prompts

Appendix C.1. Direct Judge Prompt

```
# Role
You are an evaluator assessing an RDF/Turtle knowledge graph generated from an image. Judge
     whether the graph reflects the visible source image and is well constructed as a
    knowledge graph.
# Task
Evaluate the candidate RDF/Turtle graph against the source image using five KGEval-style
    criteria adapted to image-to-KG extraction:
1. Relevance: Does the graph focus on information that is visible and important in the
    image?
2. Factuality: Are entities, labels, values, relationships, and chart or table claims
    grounded in the image without hallucination?
3. Informativeness: Does the graph capture the major useful content, such as chart titles,
    axes, data points, table rows, columns, diagram nodes, and relationships?
4. Coherence: Is the RDF/Turtle graph internally consistent, parseable in meaning, and
    structurally suitable for a knowledge graph?
5. Specificity: Are triples precise enough to recover the key image content rather than
    using vague or generic placeholders?
Also assign an ‘overall_score‘ from 1 to 5 as a holistic quality score.
## Rating Scale
- ‘1‘: unusable, unrelated, hallucinated, or incoherent.
- ‘2‘: limited correct content with major omissions, hallucinations, or graph problems.
- ‘3‘: partially correct but incomplete, noisy, or loosely aligned.
- ‘4‘: mostly faithful and useful with minor omissions or local errors.
- ‘5‘: faithful, informative, coherent, specific, and well aligned with the image.
## Evaluation Steps
1. Inspect the source image and identify its key visible content.
2. Review the RDF/Turtle graph and compare its triples against the image.
3. For each criterion, assess the graph using the rating scale and criterion guidelines.
4. Identify major errors that materially affect graph quality.
5. Return only the structured JSON object required by the response schema.
```

Appendix C.2. Pairwise Judge Prompt

```
# Role
You are an evaluator comparing two RDF/Turtle knowledge graphs generated from the same
    source image. Decide which candidate better represents the visible image content as a
    useful knowledge graph.
```

```
# Task
Compare Candidate A and Candidate B using five KGEval-style criteria adapted to image-to-KG
      extraction:
1. Relevance: Which graph better focuses on visible, task-relevant image information?
2. Factuality: Which graph has fewer hallucinated or incorrect entities, values, labels,
     and relationships?
3. Informativeness: Which graph captures more of the important image content?
4. Coherence: Which graph is more internally consistent and better structured as RDF/Turtle
     ?
5. Specificity: Which graph is more precise and less generic?
For each criterion, choose A, B, or tie. Then choose an overall winner. Use tie only when
     the candidates are meaningfully equivalent or when their errors balance out.
## Evaluation Steps
1. Inspect the source image and identify the key visible facts.
2. Analyze Candidate A and Candidate B independently.
3. Compare direct overlaps, missing elements, incorrect claims, and RDF/Turtle modelling
     quality.
4. Select a preference for each criterion.
5. Select the overall winner and confidence.
6. Return only the structured JSON object required by the response schema.
```

References

[1] R. Cyganiak, D. Wood, M. Lanthaler, RDF 1.1 Concepts and Abstract Syntax, W3c recommendation, World Wide Web Consortium (2014).
URL https://www.w3.org/TR/rdf11-concepts/

[2] D. Beckett, T. Berners-Lee, E. Prud'hommeaux, G. Carothers, RDF 1.1 Turtle: Terse RDF Triple Language, W3c recommendation, World Wide Web Consortium (2014).
URL https://www.w3.org/TR/turtle/

[3] J. Luo, Z. Li, J. Wang, C. Lin, ChartOCR: Data extraction from charts images via a deep hybrid framework, in: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, 2021, pp. 1917–1925.
URL https://openaccess.thecvf.com/content/WACV2021/html/Luo_ChartOCR_Data_Extraction_From_Charts_Images_via_a_Deep_Hybrid_WACV_2021_paper.html

[4] A. Masry, X. L. Do, J. Q. Tan, S. Joty, E. Hoque, ChartQA: A benchmark for question answering about charts with visual and logical reasoning, in: Findings of the Association for Computational Linguistics: ACL 2022, 2022, pp. 2263–2279. doi:10.18653/v1/2022.findings-acl.

177.
URL https://aclanthology.org/2022.findings-acl.177/

[5] B. J. Tang, A. Boggust, A. Satyanarayan, VisText: A benchmark for semantically rich chart captioning, in: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2023, pp. 7268–7298. doi:10.18653/v1/2023.acl-long.401.
URL https://aclanthology.org/2023.acl-long.401/

[6] A. Masry, P. Kavehzadeh, X. L. Do, E. Hoque, S. Joty, UniChart: A universal vision-language pretrained model for chart comprehension and reasoning, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023, pp. 14662–14684. doi:10.18653/v1/2023.emnlp-main.906.
URL https://aclanthology.org/2023.emnlp-main.906/

[7] Y. Han, C. Zhang, X. Chen, X. Yang, Z. Wang, G. Yu, B. Fu, H. Zhang, ChartLlama: A multimodal llm for chart understanding and generation, arXiv preprint arXiv:2311.16483 (2023).
URL https://arxiv.org/abs/2311.16483

[8] Zackriya Solutions, Diagram2Graph: Extracting structured graph data from process/flow diagrams, https://github.com/Zackriya-Solutions/diagram2graph, accessed 2026-04-20 (2025).

[9] H. Pan, Q. Zhang, C. Caragea, E. Dragut, L. J. Latecki, FlowLearn: Evaluating large vision-language models on flowchart understanding, arXiv preprint arXiv:2407.05183 (2024).
URL https://arxiv.org/abs/2407.05183

[10] M. Zheng, X. Feng, Q. Si, Q. She, Z. Lin, W. Jiang, W. Wang, Multimodal table understanding, in: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2024, pp. 9102–9124. doi:10.18653/v1/2024.acl-long.493.
URL https://aclanthology.org/2024.acl-long.493/

[11] W. Zhao, H. Feng, Q. Liu, J. Tang, S. Wei, B. Wu, L. Liao, Y. Ye, H. Liu, W. Zhou, H. Li, C. Huang, TabPedia: Towards comprehensive visual table understanding with concept synergy, arXiv preprint

arXiv:2406.01326 (2024).
URL https://arxiv.org/abs/2406.01326

[12] V. Efthymiou, E. Jiménez-Ruiz, J. Chen, V. Cutrona, O. Hassanzadeh, J. Sequeda, K. Srinivas, N. Abdelmageed, M. Hulsebos, et al., Results of SemTab 2022: Semantic web challenge on tabular data to knowledge graph matching, in: CEUR Workshop Proceedings, Vol. 3320, 2023.
URL https://ceur-ws.org/Vol-3320/paper0.pdf

[13] SemTab 2024: Semantic web challenge on tabular data to knowledge graph matching, https://sem-tab-challenge.github.io/2024/, accessed 2026-04-20 (2024).

[14] V. Nechakhin, J. D'Souza, S. Eger, S. Auer, KGEval: Evaluating scientific knowledge graphs with large language models, Information 17 (1) (2026) 35. doi:10.3390/info17010035.
URL https://www.mdpi.com/2078-2489/17/1/35

[15] J. Bayer, L. van Waveren, A. Dengel, Modular graph extraction for handwritten circuit diagram images, arXiv preprint arXiv:2402.11093 (2024).
URL https://arxiv.org/abs/2402.11093

[16] M. Wang, H. Wang, G. Qi, Q. Zheng, Richpedia: A large-scale, comprehensive multi-modal knowledge graph, Big Data Research 22 (2020) 100159. doi:10.1016/j.bdr.2020.100159.
URL https://doi.org/10.1016/j.bdr.2020.100159

[17] X. Zhu, Z. Li, X. Wang, X. Jiang, P. Sun, X. Wang, Y. Xiao, N. J. Yuan, Multi-modal knowledge graph construction and application: A survey, IEEE Transactions on Knowledge and Data Engineering 36 (2) (2024) 715–735. doi:10.1109/TKDE.2022.3224228.
URL https://doi.org/10.1109/TKDE.2022.3224228

[18] L. Zheng, W. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, I. Stoica, Judging llm-as-a-judge with MT-Bench and chatbot arena, arXiv preprint arXiv:2306.05685 (2023).
URL https://arxiv.org/abs/2306.05685

[19] K. Papineni, S. Roukos, T. Ward, W.-J. Zhu, BLEU: A method for automatic evaluation of machine translation, in: Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, 2002, pp. 311–318. doi:10.3115/1073083.1073135.
URL https://aclanthology.org/P02-1040/

[20] C. Lin, ROUGE: A package for automatic evaluation of summaries, in: Text Summarization Branches Out, 2004, pp. 74–81.
URL https://aclanthology.org/W04-1013/

[21] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, Y. Artzi, BERTScore: Evaluating text generation with BERT, in: International Conference on Learning Representations, 2020.
URL https://openreview.net/forum?id=SkeHuCVFDr